```python
import os
import time
import threading
import numpy as np
from concurrent.futures import ThreadPoolExecutor
from math import pi

SAMPLES = 40_000_000
THREADS = 4
BATCH_SIZE = 10_000
def count_inside(points):  2 usages
    return int(np.count_nonzero(
        points[:, 0] * points[:, 0] +
        points[:, 1] * points[:, 1] <= 1.0
    ))
def shared_prng_simulation():  1 usage
    rng = np.random.default_rng(42)
    lock = threading.Lock()
    counts = [0] * THREADS
    samples_per_thread = SAMPLES // THREADS
    def worker(index):
        inside = 0
        remaining = samples_per_thread
        while remaining > 0:
            batch = min(BATCH_SIZE, remaining)
            with lock:
                points = rng.random((batch, 2))

            inside += count_inside(points)
            remaining -= batch

```

```
16      def shared_prng_simulation():  1 usage
21          def worker(index):
30                  remaining -= batch
31
32              counts[index] = inside
33
34          start = time.perf_counter()
35
36          with ThreadPoolExecutor(max_workers=THREADS) as executor:
37              list(executor.map(worker, *iterables: range(THREADS)))
38
39          elapsed_ms = (time.perf_counter() - start) * 1000
40          total_inside = sum(counts)
41          estimate = 4.0 * total_inside / SAMPLES
42
43          return estimate, abs(estimate - pi), elapsed_ms
44
45
46      def independent_prng_simulation(student_id):  1 usage
47          counts = [0] * THREADS
48          samples_per_thread = SAMPLES // THREADS
49
50          def worker(index):
51              seed = student_id * 10 + index
52              rng = np.random.default_rng(seed)
53              inside = 0
54              remaining = samples_per_thread
55
56              while remaining > 0:
57                  batch = min(BATCH_SIZE, remaining)
58                  points = rng.random((batch, 2))
```

```
46      def independent_prng_simulation(student_id):  1 usage
50          def worker(index):
57                  batch = min(BATCH_SIZE, remaining)
58                  points = rng.random((batch, 2))
59                  inside += count_inside(points)
60                  remaining -= batch
61
62              counts[index] = inside
63
64          start = time.perf_counter()
65
66          with ThreadPoolExecutor(max_workers=THREADS) as executor:
67              list(executor.map(worker,  *iterables: range(THREADS)))
68
69          elapsed_ms = (time.perf_counter() - start) * 1000
70          total_inside = sum(counts)
71          estimate = 4.0 * total_inside / SAMPLES
72
73          return estimate, abs(estimate - pi), elapsed_ms
74
75
76      def main():  1 usage
77          raw_id = input("Enter your numeric Student ID: ").strip()
78
79          try:
80              student_id = int(raw_id)
81              if student_id < 0:
82                  raise ValueError
83          except ValueError:
84              print("Please enter a non-negative numeric Student ID.")
85              return
```

```python
def main():  1 usage

    print("\nTASK 2 — PRNG State Bottleneck")
    print(f"Student ID: {student_id}")
    print(f"Logical CPU count: {os.cpu_count()}")
    print(f"Sample points: {SAMPLES:,}")
    print(f"Threads: {THREADS}")
    shared_pi, shared_error, shared_ms = shared_prng_simulation()
    independent_pi, independent_error, independent_ms = (
        independent_prng_simulation(student_id)
    )
    speedup = (
        shared_ms / independent_ms
        if independent_ms > 0
        else float("inf")
    )
    lines = [
        "TASK 2 — PRNG State Bottleneck",
        f"Student ID: {student_id}",
        f"Logical CPU count: {os.cpu_count()}",
        f"Sample points: {SAMPLES}",
        f"Threads: {THREADS}",
        "",
        f"Shared PRNG computed pi: {shared_pi:.10f}",
        f"Shared PRNG absolute error: {shared_error:.10f}",
        f"Shared PRNG wall time: {shared_ms:.2f} ms",
        "",
        f"Independent PRNG computed pi: {independent_pi:.10f}",
        f"Independent PRNG absolute error: {independent_error:.10f}",
        f"Independent PRNG wall time: {independent_ms:.2f} ms",
        "",
```

```
        f"Independent PRNG wall time: {independent_ms:.2f} ms",
        "",
        f"Speedup factor: {speedup:.2f}x",
    ]

    output = "\n".join(lines)
    print("\n" + output)

    with open("task2_output.txt", "w", encoding="utf-8") as file:
        file.write(output + "\n")

    print("\nResults saved to task2_output.txt")


if __name__ == "__main__":
    main()
```